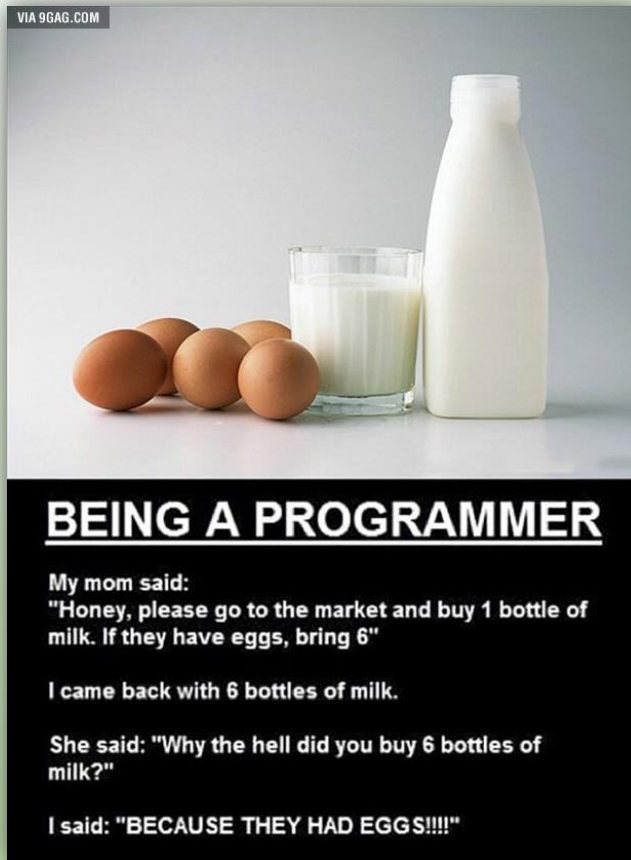


Chapter 5:

Loops





Objectives

- To write programs for executing statements repeatedly using a **while** loop (§5.2).
- To follow the loop design strategy to develop loops (§§5.2.1–5.2.3).
- To control a loop with a sentinel value (§5.2.4).
- To obtain large input from a file using input redirection rather than typing from the keyboard (§5.2.5).
- To write loops using **do-while** statements (§5.3).
- To write loops using **for** statements (§5.4).
- To discover the similarities and differences of three types of loop statements (§5.5).
- To write nested loops (§5.6).
- To learn the techniques for minimizing numerical errors (§5.7).
- To learn loops from a variety of examples (**GCD**, **FutureTuition**, **Dec2Hex**) (§5.8).
- To implement program control with **break** and **continue** (§5.9).
- To write a program that displays prime numbers (§5.11).



Motivations

- ♦ What if you wanted to print the same statement 100 times?
 - Example: print "Welcome to Java" one hundred times

```
100 times { System.out.println("Welcome to Java!");  
           System.out.println("Welcome to Java!");  
           ...  
           System.out.println("Welcome to Java!");
```

- Certainly, there **MUST** be a better way to do this!



Motivations

♦ Solution:

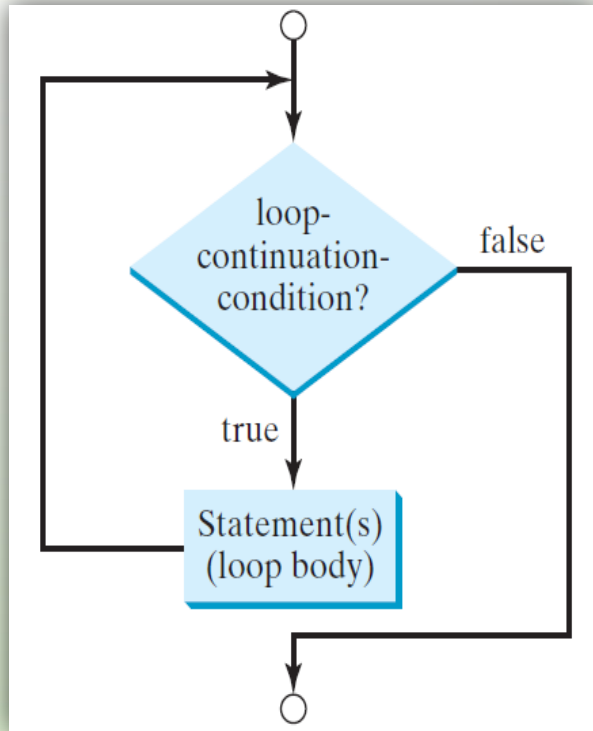
- Java provides a powerful construct called a *loop*
- A *loop* controls how many times an operation (or a sequence of operations) is performed
- You can use a loop statement to tell the computer to display a string one hundred times



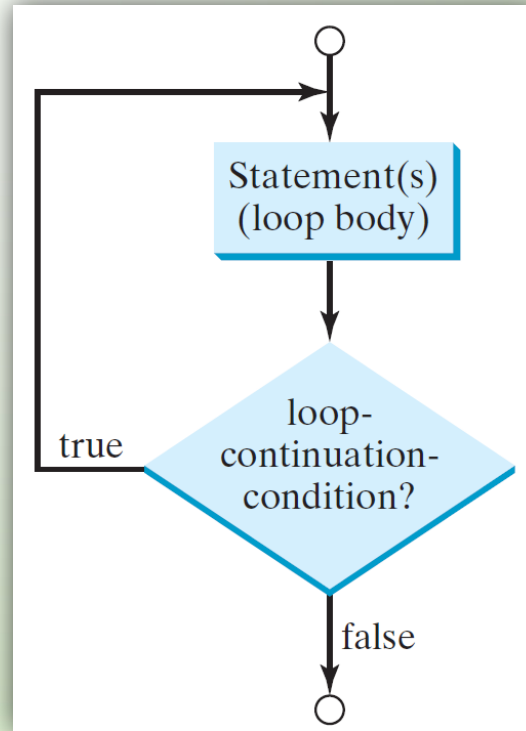
Loops

- ◆ Loops are constructs that allow repeated execution of a block of statements
- ◆ Understanding and using loops is fundamental to programming
- ◆ Java provides three types of loop statements:
 1. **while** loops
 2. **do-while** loops
 3. **for** loops

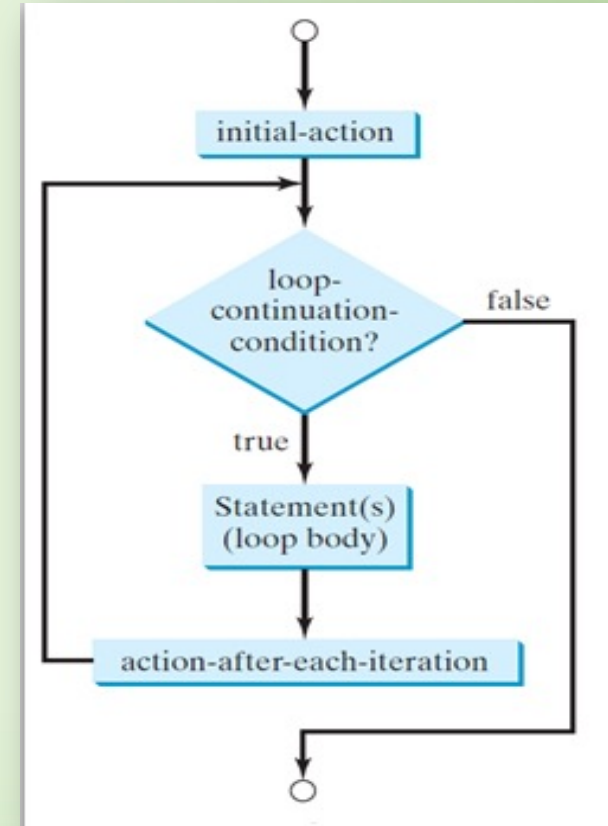
Three Types of Loops



while Loop



do-while Loop



for Loop

We will study each in detail....



Loop Design Strategies

- ♦ Coding a correct loop is challenging for new programmers
- ♦ Therefore, three steps are recommended:
 - Step 1: identify the statements that need to be repeated
 - Step 2: Wrap these statements in any of the three types loop
 - Step 3: Code the loop-continuation-condition and include appropriate statements to control the loop



Loop Design Strategies

♦ Note:

- Make sure the loop-continuation-condition will eventually become false
 - ♦ This will allow the loop to terminate
- If your loop is running a very long time and does not want to stop, you most likely have an infinite loop
- In NetBeans, click the small "x" in the bottom right corner
 - ♦ This will stop the execution of the program



The **while** Loop

✦ Syntax:

```
while (loop-continuation-condition) {  
    // loop body  
    Statement(s);  
}
```

- Each loop contains a loop-continuation-condition
- This is a Boolean expression that controls the execution of the loop body
- This expression is evaluated at EACH iteration
 - ◆ If the result is **true**, the loop body is executed
 - ◆ If it is **false**, the entire loop will terminate
 - program control goes to the next statement after the loop



The **while** Loop

◆ Syntax:

```
while (loop-continuation-condition) {  
    // loop body  
    Statement(s);  
}
```

- the "loop body" is the part that contains the repeated statements
- a one-time execution of the loop body is called an *iteration*



The **while** Loop

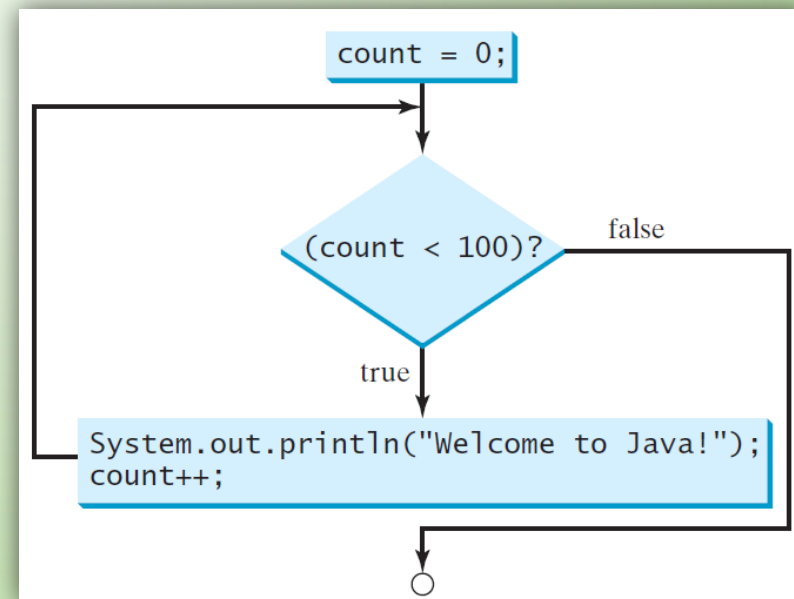
- ◆ Print "Welcome to Java" one hundred times:

```
int count = 0;
while (count < 100)
    System.out.println("Welcome to Java!");
    count++;
}
```

This type of loop is called a **counter**-controlled loop.

– Flowchart:

- ◆ The continuation condition is $\text{count} < 100$
- ◆ If **true**, the loop continues
- ◆ If **false**, the loop will terminate





Trace while Loop

```
int count = 0;
```

Initialize count

```
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

Trace while Loop, cont.

```
int count = 0;
```

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

```
}
```

(count < 2) is true

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

Print Welcome to Java

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

Increase count by 1
count is 1 now



Trace while Loop, cont.

```
int count = 0;
while (count < 2) {
    System.out.println("Welcome to Java!");
    count++;
}
```

(count < 2) is still true
because count is 1

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```



Print Welcome to Java

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

Increase count by 1
count is 2 now



Trace while Loop, cont.

```
int count = 0;
```

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

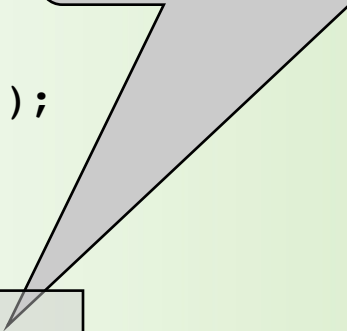
```
    count++;
```

```
}
```

(count < 2) is false
because count is 2 now

Trace while Loop

```
int count = 0;
while (count < 2) {
    System.out.println("Welcome to Java!");
    count++;
}
```



The loop exits. Execute the next statement after the loop.





The **while** Loop

◆ Another example:

```
int sum = 0, i = 1;
while (i < 10) {
    sum = sum + 1;
    i++;
}
System.out.println("sum is " + sum); // sum is 45
```

– Details:

- ◆ if **i < 10** is **true**, the program adds **i** to **sum**
- ◆ variable **i** is initially set to **1**
 - then it is incremented to **2**, **3**, and up to **10**
- ◆ When **i** is **10**, **i < 10** is **false**, and the loop will exit
- ◆ So the sum is **1 + 2 + 3 + ... + 9 = 45**



The **while** Loop

♦ What happens if we forget the `i++` operation?

```
int sum = 0, i = 1;
while (i < 10) {
    sum = sum + 1;
}
System.out.println("sum is " + sum); // sum is 45
```

– Details:

- ♦ variable **i** starts at **1**
- ♦ and it never gets bigger than 1
- ♦ and 1 is always smaller than 10
- ♦ The loop is infinite!
 - because **i** is always **1** and **i < 10** will always be **true**



The **while** Loop

♦ Caution:

- New programmers often make the mistake of executing a loop one extra time (or one less time)
- Example:

```
int count = 0;
while (count <= 100)
    System.out.println("Welcome to Java!");
    count++;
}
```

♦ This displays "Welcome to Java" 101 times

♦ Why?

- **count** starts at **0**, which means it should go until **count < 100**
- If you want to iterate until **count <= 100**, then start count at **1**

Check Point

♦ How many times are the following loop bodies repeated? What is the output of each?

```
int i = 1;
while (i < 10)
    if (i % 2 == 0)
        System.out.println(i);
```

(a)

```
int i = 1;
while (i < 10)
    if (i % 2 == 0)
        System.out.println(i++);
```

(b)

```
int i = 1;
while (i < 10)
    if ((i++) % 2 == 0)
        System.out.println(i);
```

(c)



Program 1: Addition Quiz

- ◆ Remember we wrote a program to generate two numbers randomly and then ask the user for the answer of $\text{num1} + \text{num2}$. Now, we rewrite this program to let the user repeatedly enter a new answer until it is correct.
- ◆ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 1: Addition Quiz

♦ Step 1: Problem-solving Phase

– Algorithm:

- ♦ Generate two random numbers
 - Use `(int)(Math.random() * 10)`
- ♦ Ask the user for the answer and scan answer
- ♦ Make a while loop
 - Condition of loop is if the answer is wrong
 - Because if it is wrong, we should ask again
 - and we should scan again
- ♦ Once the answer is finally correct:
 - Exit the loop
 - Print "You got it!"



Program 1: Addition Quiz

♦ Step 2: Implementation

LISTING 5.1 RepeatAdditionQuiz.java

```
1  import java.util.Scanner;
2
3  public class RepeatAdditionQuiz {
4      public static void main(String[] args) {
5          int number1 = (int)(Math.random() * 10);
6          int number2 = (int)(Math.random() * 10);
7
8          // Create a Scanner
9          Scanner input = new Scanner(System.in);
10
11         System.out.print(
12             "What is " + number1 + " + " + number2 + "? ";
13         int answer = input.nextInt();
14
15         while (number1 + number2 != answer) {
16             System.out.print("Wrong answer. Try again. What is "
17                 + number1 + " + " + number2 + "? ");
18             answer = input.nextInt();
19         }
20
21         System.out.println("You got it!");
22     }
23 }
```

Program 1: Addition Quiz

♦ Run the Program:



What is 5 + 9? 12

Wrong answer. Try again. What is 5 + 9? 34

Wrong answer. Try again. What is 5 + 9? 14

You got it!

[Click here](#) to view and trace code



Check Point

- ♦ What is the output of the following program?
Suppose the input is **2 3 4 5 0**:

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int number, max;
        number = input.nextInt();
        max = number;
        while (number != 0) {
            number = input.nextInt();
            if (number > max)
                max = number;
        }
        System.out.println("max is " + max);
        System.out.println("number " + number);
    }
}
```



Program 2: Guessing Game

- ♦ Write a program that randomly generates a number between 0 and 100, inclusive. The program will repeatedly ask the user to guess the number until the user gets the number correct. At each wrong answer, the program tells the user if the guess is too low or too high.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase

Program 2: Guessing Game

♦ Example run of program:

```
Guess a magic number between 0 and 100
Enter your guess: 50 ↵ Enter
Your guess is too high
Enter your guess: 25 ↵ Enter
Your guess is too low
Enter your guess: 42 ↵ Enter
Your guess is too high
Enter your guess: 39 ↵ Enter
Yes, the number is 39
```





Program 2: Guessing Game

♦ Step 1: Problem-solving Phase

- So are we ready to code?
 - ♦ NO!
- We must THINK before coding.
- Think:
 - ♦ How would you solve the problem without a program?
 - ♦ You need a random number between 0 and 100
 - ♦ You need to ask the user to enter a guess
 - ♦ You need to compare the guess with the random number



Program 2: Guessing Game

♦ Step 1: Problem-solving Phase

- Good idea to code *incrementally* when using loops
- Meaning:
 - ♦ Do not write the looping structure immediately
 - ♦ First, try to just write the logic of the program, but without using loops
 - So just write the code for one "loop", one iteration
 - ♦ Then, write the code for the loop structure
 - ♦ Think of the following code as an "initial draft"



Program 2: Guessing Game

♦ Step 2: Implementation (Initial draft w/o Loops)

LISTING 5.2 GuessNumberOneTime.java

```
1  import java.util.Scanner;
2
3  public class GuessNumberOneTime {
4      public static void main(String[] args) {
5          // Generate a random number to be guessed
6          int number = (int)(Math.random() * 101);
7
8          Scanner input = new Scanner(System.in);
9          System.out.println("Guess a magic number between 0 and 100");
10
11         // Prompt the user to guess the number
12         System.out.print("\nEnter your guess: ");
13         int guess = input.nextInt();
14
15         if (guess == number)
16             System.out.println("Yes, the number is " + number);
17         else if (guess > number)
18             System.out.println("Your guess is too high");
19         else
20             System.out.println("Your guess is too low");
21     }
22 }
```



Program 2: Guessing Game

♦ Step 1: Problem-solving Phase

- The previous code only allowed one guess
- So now we need to use a loop construct to allow many guesses
- When the guess is finally correct, the loop will exit
- So what is the loop condition?

while (guess != number)

- ♦ So if the guess is **not** the same as the random number, continue the while loop



Program 2: Guessing Game

♦ Step 2: Implementation

LISTING 5.3 GuessNumber.java

```
1  import java.util.Scanner;
2
3  public class GuessNumber {
4      public static void main(String[] args) {
5          // Generate a random number to be guessed
6          int number = (int)(Math.random() * 101);
7
8          Scanner input = new Scanner(System.in);
9          System.out.println("Guess a magic number between 0 and 100");
10
11         int guess = -1;
12         while (guess != number) {
13             // Prompt the user to guess the number
14             System.out.print("\nEnter your guess: ");
15             guess = input.nextInt();
16
17             if (guess == number)
18                 System.out.println("Yes, the number is " + number);
19             else if (guess > number)
20                 System.out.println("Your guess is too high");
21             else
22                 System.out.println("Your guess is too low");
23         } // End of loop
24     }
25 }
```

Program 2: Guessing Game

♦ Run the Program:

```
Guess a magic number between 0 and 100
Enter your guess: 50 ↵ Enter
Your guess is too high
Enter your guess: 25 ↵ Enter
Your guess is too low
Enter your guess: 42 ↵ Enter
Your guess is too high
Enter your guess: 39 ↵ Enter
Yes, the number is 39
```



[Click here](#) to view and trace code



Caution

- ◆ Do not use floating-point values for equality checking inside the loop condition
 - Why?
 - Floating-point values are approximate
 - Example:

```
double item = 1; double sum = 0;
while (item != 0) { // no guarantee item will be 0
    sum += item;
    item -= 0.1;
}
System.out.println(sum);
```

- ◆ Here, **item** starts at **1** and is reduced by **0.1** each time
- ◆ The loop **should** terminate when $\text{item} = 0$...but it will NOT



The **do-while** Loop

♦ Syntax:

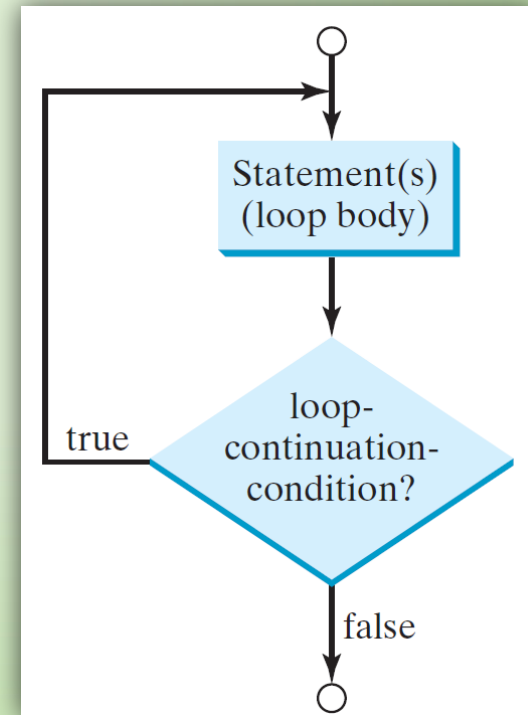
```
do {  
    // loop body  
    Statement(s);  
} while (loop-continuation-condition);
```

- ♦ The **do-while** loop is very similar to the while loop
- ♦ Difference:
 - The **do-while** loop executes the loop body first
 - Then it checks the loop continuation condition



The **do-while** Loop

- ♦ So for a **do-while** loop, the body is executed first, and then the condition is evaluated
 - If **true**, the loop body executes again
 - If **false**, the **do-while** loop will terminate





Check Point

- ♦ What is the output of the following program?
Suppose the input is **2 3 4 5 0**:

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int number, max;
        number = input.nextInt();
        max = number;
        do {
            number = input.nextInt();
            if (number > max)
                max = number;
        } while (number != 0);
        System.out.println("max is " + max);
        System.out.println("number " + number);
    }
}
```



Controlling a Loop with a Sentinel Value

♦ Common technique:

- Control the loop by choosing a special value when reading and processing user input
- This special input value is known as a *sentinel value*
- The sentinel value signifies the end of the input
- Example:
 - ♦ ask the user to keep inputting as many integer values as they want
 - ♦ Tell them that the loop will stop once the value -1 is entered
 - ♦ Therefore, -1 would be the sentinel value



Sentinel Value Program

- ◆ Write a program that will sum up all user inputted values. The user can keep inputting values for as long as they wish. If the user enters "0", this means the end of the input. Your program should display the sum to the user.
- ◆ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Sentinel Value Program

♦ Step 1: Problem-solving Phase

- Use the loop design strategy
 - ◆ First, identify the statements that need to be repeated
 - Ask the user for a value
 - Scan the value
 - Add it to the variable sum
 - ◆ Second, wrap these statements inside a loop
 - ◆ Finally, add a loop control variable and an appropriate loop-continuation-condition that will execute the loop five times



Sentinel Value Program

LISTING 5.5 SentinelValue.java

```
1  import java.util.Scanner;
2
3  public class SentinelValue {
4      /** Main method */
5      public static void main(String[] args) {
6          // Create a Scanner
7          Scanner input = new Scanner(System.in);
8
9          // Read an initial data
10         System.out.print(
11             "Enter an integer (the input ends if it is 0): ");
12         int data = input.nextInt();
13
14         // Keep reading data until the input is 0
15         int sum = 0;
16         while (data != 0) {
17             sum += data;
18
19             // Read the next data
20             System.out.print(
21                 "Enter an integer (the input ends if it is 0): ");
22             data = input.nextInt();
23         }
24
25         System.out.println("The sum is " + sum);
26     }
27 }
```

Program 4: Sentinel Value

♦ Run the Program:



```
Enter an integer (the input ends if it is 0): 2 ↵Enter
Enter an integer (the input ends if it is 0): 3 ↵Enter
Enter an integer (the input ends if it is 0): 4 ↵Enter
Enter an integer (the input ends if it is 0): 0 ↵Enter
The sum is 9
```

[Click here](#) to view and trace code



Sentinel Value Program (v2)

- ◆ Write a program that will sum up all user inputted values. The user can keep inputting values for as long as they wish. If the user enters "0", this means the end of the input. Your program should display the sum to the user.
- ◆ Use a do-while loop for this program.
- ◆ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Sentinel Value Program (v2)

♦ Step 1: Problem-solving Phase

- Use the loop design strategy
 - ♦ Since we are using a **do-while** loop, this means we will "do" the repeated actions at least one time
 - Because there is no condition upon entering the loop
 - ♦ First, identify the statements that need to be repeated
 - Ask the user for a value
 - Scan the value
 - Add it to the variable **sum**
 - ♦ Second, wrap these statements inside a loop
 - ♦ Finally, add the loop condition to the **while** component of the **do-while** loop



Sentinel Value Program (v2)

LISTING 5.6 TestDoWhile.java

```
1  import java.util.Scanner;
2
3  public class TestDoWhile {
4      /** Main method */
5      public static void main(String[] args) {
6          int data;
7          int sum = 0;
8
9          // Create a Scanner
10         Scanner input = new Scanner(System.in);
11
12         // Keep reading data until the input is 0
13         do {
14             // Read the next data
15             System.out.print(
16                 "Enter an integer (the input ends if it is 0): ");
17             data = input.nextInt();
18
19             sum += data;
20         } while (data != 0);
21
22         System.out.println("The sum is " + sum);
23     }
24 }
```

Sentinel Value Program (v2)

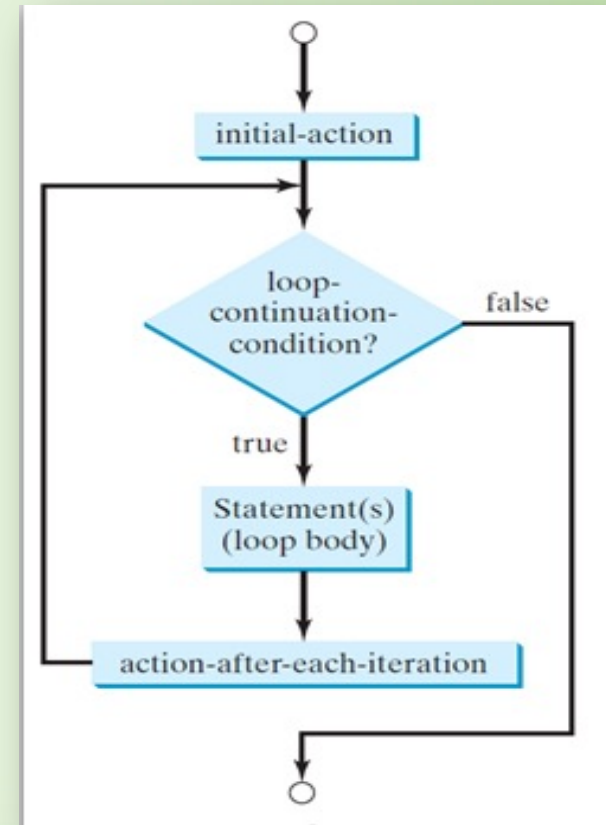
♦ Run the Program:

```
Enter an integer (the input ends if it is 0): 3 ↵ Enter
Enter an integer (the input ends if it is 0): 5 ↵ Enter
Enter an integer (the input ends if it is 0): 6 ↵ Enter
Enter an integer (the input ends if it is 0): 0 ↵ Enter
The sum is 14
```



The **for** Loop

- ♦ If you want to iterate a specific number of times, it is better/easier to just use a **for** loop
- ♦ In general, the syntax of a **for** loop is:



```
for (initial-action; loop-condition; action-after) {  
    // Loop body  
    Statement(s);  
}
```



The **for** Loop

◆ Details:

- The **for** loop statements starts with the keyword **for**
- Next is a pair of parentheses enclosing the control structure of the loop
- The loop control structure consists of three parts:
 1. initial-action
 2. loop-continuation-condition
 3. action-after-each-iteration
- The control structure is then followed by the body of the loop



The **for** Loop

◆ Details:

- A **for** loop usually uses a variable to control how many times the loop body is executed
- This variable also controls when the loop terminates
- This variable is called the **control variable**
- The **initial-action** often initializes the control variable
- The **action-after-each-iteration** usually increments or decrements the control variable
- And the **loop-continuation-condition** tests whether the control variable has reached a termination value



The **for** Loop

◆ Example:

- Print "Welcome to Java" one hundred times

```
for (int i = 0; i < 100; i++) {  
    System.out.println("Welcome to Java!");  
}
```

– Details:

- ◆ The initial-action declares and initializes variable **i**
- ◆ The loop-continuation-condition is evaluated right after the initialization AND at the beginning of each loop iteration
- ◆ If **true**, the body is executed (we print)
- ◆ If **false**, the loop will terminate



The **for** Loop

◆ Example:

- Print "Welcome to Java" one hundred times

```
for (int i = 0; i < 100; i++) {  
    System.out.println("Welcome to Java!");  
}
```

– Details:

- ◆ The action-after-each-iteration (**i++**) is a statement that will adjust the control variable (**i**)
- ◆ This statement is executed after each iteration of the loop
- ◆ In this example, the control variable is incremented
- ◆ Eventually, the loop-continuation-condition will be **false**



Trace for Loop

```
int i;
```

Declare i

```
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute initializer
i is now 0

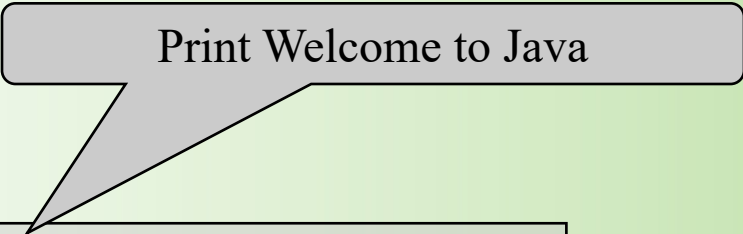
Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

(i < 2) is true
since i is 0

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```



Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute increment statement
i now is 1

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

(i < 2) is still true
since i is 1

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Print Welcome to Java

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute adjustment statement
i now is 2

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

($i < 2$) is false
since i is 2

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Exit the loop. Execute the next statement after the loop





The **for** Loop

♦ Note:

- The initial-action in a for loop can be a list of zero or more comma-separated declaration statements or assignment expressions

```
for (int i = 0, j = 0; i + j < 10; i++, j++) {  
    // Do something  
}
```

- The action-after-each-iteration in a for loop can also be a list of zero or more comma-separated statements

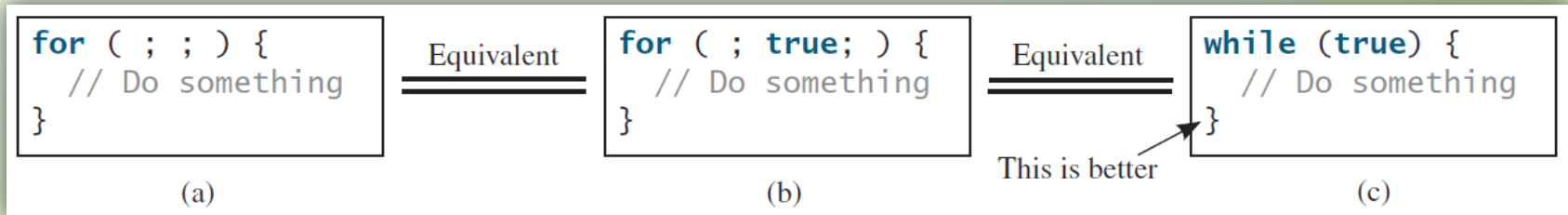
```
for (int i = 1; i < 100; System.out.println(i), i++) {  
    // Do something  
}
```



The **for** Loop

♦ Note:

- The loop-continuation-condition can be omitted
- If it is omitted, it is implicitly true
 - ♦ Meaning, it is understood as being true!
- Example:



- ♦ (a) and (b) are equivalent in meaning
- ♦ but using (c) is better in this example

Caution

♦ Common mistakes:

- Adding a semicolon at the end of the for clause

Error

```
for (int i = 0; i < 10; i++);  
{  
    System.out.println("i is " + i);  
}
```

(a)

Empty body

```
for (int i = 0; i < 10; i++) { };  
{  
    System.out.println("i is " + i);  
}
```

(b)

Error

```
int i = 0;  
while (i < 10);  
{  
    System.out.println("i is " + i);  
    i++;  
}
```

(c)

Empty body

```
int i = 0;  
while (i < 10) { };  
{  
    System.out.println("i is " + i);  
    i++;  
}
```

(d)



Check Point

- ♦ What is the output of the following program?
Suppose the input is **2 3 4 5 0**:

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int number, sum = 0, count;

        for (count = 0; count < 5; count++) {
            number = input.nextInt();
            sum += number;
        }

        System.out.println("sum is " + sum);
        System.out.println("count is " + count);
    }
}
```



Check Point

- ◆ Convert the following **for** loop into a **while** loop and into a **do-while** loop:

```
long sum = 0;
for (int i = 0; i <= 1000; i++) {
    sum = sum + i;
}
```



Check Point

♦ What is wrong with the following programs:

```
1 public class ShowErrors {  
2     public static void main(String[] args) {  
3         int i = 0;  
4         do {  
5             System.out.println(i + 4);  
6             i++;  
7         }  
8         while (i < 10)  
9     }  
10 }
```

(a)

```
1 public class ShowErrors {  
2     public static void main(String[] args) {  
3         for (int i = 0; i < 10; i++);  
4             System.out.println(i + 4);  
5     }  
6 }
```

(b)

a) no semicolon after the while statement

♦ you need a semicolon here

b) semicolon used at end of for clause

♦ You cannot use a semicolon here



Keywords **break** and **continue**

- ◆ We can use two keywords,
 - **break** and **continue**,
- ◆ to provide additional control inside our loops
- ◆ Benefits of using these keywords:
 - Can simplify programming (sometimes)
- ◆ Negatives of using these keywords:
 - Overuse or improperly using them can cause problems and make programs difficult to read and debug



Keywords **break** and **continue**

♦ **break**:

- allows you to immediately terminate a loop
- think of this as similar to a break in a switch statement

♦ **continue**:

- ends the current iteration of the loop and program control skips until the end of the loop body

♦ Summary:

- **break** breaks out of the loop
- **continue** breaks out of the iteration



Keywords **break** and **continue**

♦ Caution:

- In **while** and **do-while** loops, as soon as the keyword **continue** is executed, program control skips to the end of the loop body and then the loop-continuation-condition is evaluated
- In **for** loops, as soon as the keyword **continue** is executed, program control skips to the end of the loop body, and then the action-after-each-iteration and then the loop-continuation-condition is evaluated
- The difference is the action-after-each-iteration is executed with a for loop and not with while/do-while



Keywords **break** and **continue**

◆ **break**:

[Click here](#)
to view and
trace code

LISTING 5.12 TestBreak.java

```
1  public class TestBreak {
2      public static void main(String[] args) {
3          int sum = 0;
4          int number = 0;
5
6          while (number < 20) {
7              number++;
8              sum += number;
9              if (sum >= 100)
10                 break;
11          }
12
13          System.out.println("The number is " + number);
14          System.out.println("The sum is " + sum);
15      }
16  }
```



The number is 14
The sum is 105

Keywords **break** and **continue**

◆ **continue**:

[Click here](#)
to view and
trace code

LISTING 5.13 TestContinue.java

```
1  public class TestContinue {
2      public static void main(String[] args) {
3          int sum = 0;
4          int number = 0;
5
6          while (number < 20) {
7              number++;
8              if (number == 10 || number == 11)
9                  continue;
10             sum += number;
11         }
12
13         System.out.println("The sum is " + sum);
14     }
15 }
```



The sum is 189





Check Point

- ♦ In the examples below, what is the keyboard **break** for? What is the keyword **continue** for? Do the programs terminate? If so, what is the output?

```
int balance = 10;
while (true) {
    if (balance < 9)
        break;
    balance = balance - 9;
}
```

```
System.out.println("Balance is "
    + balance);
```

(a)

```
int balance = 10;
while (true) {
    if (balance < 9)
        continue;
    balance = balance - 9;
}
```

```
System.out.println("Balance is "
    + balance);
```

(b)



Check Point

- ♦ The **for** loop on the left is converted into the **while** loop on the right. What is wrong? Fix it!

```
int sum = 0;
for (int i = 0; i < 4; i++) {
    if (i % 3 == 0) continue;
    sum += i;
}
```

Converted
Wrong conversion

```
int i = 0, sum = 0;
while (i < 4) {
    if (i % 3 == 0) continue;
    sum += i;
    i++;
}
```



Nested Loops

- ♦ A loop can be nested inside another loop
- ♦ Nested loops consist of an outer (outside) loop and one or more inner loops
- ♦ Example:

```
for (int i = 0; i < 100; i++) {  
    for (int j = 0; j < 100; j++) {  
        // some code here  
    }  
}
```



Program 6: Multiplication Table

- ◆ Write a program that uses nested for loops to print out the 1 through 9 multiplication table.
- ◆ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Program 6: Multiplication Table

♦ Step 1: Problem-solving Phase

– Example of expected output:

Multiplication Table									
	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81



Program 6: Multiplication Table

♦ Step 1: Problem-solving Phase

- Examine the table:
 - ♦ We have nine rows of data
 - ♦ We have nine columns of data
- Look at the individual rows:
 - ♦ The first row contains the answer of 1×1 , 1×2 , $1 \times 3 \dots$
 - ♦ The second row contains the answer of 2×1 , 2×2 , $2 \times 3 \dots$
 - ♦ ...
 - ♦ The ninth row contains the answer of 9×1 , 9×2 , $9 \times 3 \dots$
- For each row heading (1, 2, 3, 4, 5, 6, 7, 8, 9)
 - ♦ We have that number multiplied by each of 1 through 9



Program 6: Multiplication Table

♦ Step 1: Problem-solving Phase

– Use two for loops:

1. The outer for loop will iterate over all rows

♦ And it will start at **i = 1**

– because the first row is labeled as 1

♦ And it will iterate until and including **i = 9** (the last row)

2. Then, for EACH row, the inner for loop will calculate that row's answer of the row # times the values 1 through 9



Program 6: Multiplication Table

♦ Step 2: Implementation

- Start by printing the header:

```
System.out.println(" Multiplication Table");

// Display the number title
System.out.print("      ");
for (int j = 1; j <= 9; j++)
    System.out.print("      " + j);
System.out.println();

System.out.println("-----");
```

- Output:

```
      Multiplication Table
      1   2   3   4   5   6   7   8   9
-----
```



Program 6: Multiplication Table

♦ Step 2: Implementation

- Now use a nested for loop to display the table body:

```
// for EACH row
for (int i = 1; i <= 9; i++) {
    // Print the row header
    // So print the row # plus a " | " after the number
    System.out.print(i + " | ");

    // Now, for EACH column of the row
    for (int j = 1; j <= 9; j++) {
        // Display the product (i*j) and align properly
        System.out.printf("%4d", i * j);
    }
    System.out.println();
}
```



Program 6: Multiplication Table

♦ Step 2: Implementation (Full Program)

LISTING 5.7 MultiplicationTable.java

```
1 public class MultiplicationTable {
2     /** Main method */
3     public static void main(String[] args) {
4         // Display the table heading
5         System.out.println("          Multiplication Table");
6
7         // Display the number title
8         System.out.print(" ");
9         for (int j = 1; j <= 9; j++)
10             System.out.print(" " + j);
11
12         System.out.println("\n-----");
13
14         // Display table body
15         for (int i = 1; i <= 9; i++) {
16             System.out.print(i + " | ");
17             for (int j = 1; j <= 9; j++) {
18                 // Display the product and align properly
19                 System.out.printf("%4d", i * j);
20             }
21             System.out.println();
22         }
23     }
24 }
```



Program 6: Multiplication Table

♦ Run the Program:

Multiplication Table									
	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

[Click here](#) to view and trace code



Check Point

- ◆ How many times is the **println** statement executed:

```
for (int i = 0; i < 10; i++)  
    for (int j = 0; j < i; j++)  
        System.out.println(i * j)
```

- Solution:

- ◆ The outer loop runs 10 times
 - From 0 to 9
- ◆ For each iteration, the inner loop runs from 0 to i
 - First time i is 0, then i is 1, then 2, then 3, until i is 9
- ◆ Answer: $1 + 2 + 3 + \dots + 9 = 45$ times



Check Point

◆ Show the output of the following program:

```
public class Test {  
    public static void main(String[] args) {  
        for (int i = 1; i < 5; i++) {  
            int j = 0;  
            while (j < i) {  
                System.out.print(j + " ");  
                j++;  
            }  
        }  
    }  
}
```

Program Trace

i	j
1	0
2	0
2	1
3	0
3	1
3	2
4	0
4	1
4	2
4	3

◆ Output:

0 0 1 0 1 2 0 1 2 3



Check Point

◆ Show the output of the following program:

```
public class Test {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i < 5) {  
            for (int j = i; j > 1; j--)  
                System.out.print(j + " ");  
            System.out.println("****");  
            i++;  
        }  
    }  
}
```

Program Trace

i	j
0	0
1	1
2	2
3	3
3	2
4	4
4	3
4	2

◆ Output:

2 ****

3 2 ****

4 3 2 ****



Check Point

◆ Show the output of the following program:

```
public class Test {  
    public static void main(String[] args) {  
        int i = 5;  
        while (i >= 1) {  
            int num = 1;  
            for (int j = 1; j <= i; j++) {  
                System.out.print(num + "xxx");  
                num *= 2;  
            }  
            System.out.println();  
            i--;  
        }  
    }  
}
```

Program Trace

i	i
5	1
5	2
5	3
5	4
5	5
4	1
4	2
4	3
4	4
3	1
3	2
3	3
2	1
2	2
1	1

◆ Output:

```
1xxx2xxx4xxx8xxx16xxx  
1xxx2xxx4xxx8xxx  
1xxx2xxx4xxx  
1xxx2xxx  
1xxx
```



Check Point

◆ Show the output of the following program:

```
public class Test {  
    public static void main(String[] args) {  
        int i = 1;  
        do {  
            int num = 1;  
            for (int j = 1; j <= i; j++) {  
                System.out.print(num + "G");  
                num += 2;  
            }  
            System.out.println();  
            i++;  
        } while (i <= 5);  
    }  
}
```

Program Trace

i	j
1	1
2	1
2	2
3	1
3	2
3	3
4	1
4	2
4	3
4	4
5	1
5	2
5	3
5	4
5	5

◆ Output:

1G
1G3G
1G3G5G
1G3G5G7G
1G3G5G7G9G



Which Loop to Use?

♦ Short answer:

- Use the one you feel most comfortable with!
- The **while** loop and **for** loop are called **pretest loops**
 - ♦ because the continuation-condition is checked before the loop body is executed
- The **do-while** loop is called a **posttest loop**
 - ♦ because the condition is checked after the loop body is executed
- All three loop types are equivalent and accomplish the same goals



Which Loop to Use?

♦ Summary:

- A **for** loop is often useful when the number of repetitions is known in advance
 - ♦ Example: print a message 100 times
- A **while** loop is useful if the number of repetitions is not fixed
 - ♦ Example: print until something is true (or false)
- A **do-while** loop is useful if the loop body has to be executed before the continuation-condition is tested for the first time



Subtraction Quiz Program

- ♦ Write a program to randomly generate five subtraction questions and ask user for the answer to each. Count how many the user got correct. And also display the total time spent, by the user, answering the five questions.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Subtraction Quiz Program

LISTING 5.4 SubtractionQuizLoop.java

```
1  import java.util.Scanner;
2
3  public class SubtractionQuizLoop {
4      public static void main(String[] args) {
5          final int NUMBER_OF_QUESTIONS = 5; // Number of questions
6          int correctCount = 0; // Count the number of correct answers
7          int count = 0; // Count the number of questions
8          long startTime = System.currentTimeMillis();
9          String output = " "; // output string is initially empty
10         Scanner input = new Scanner(System.in);
11
12         while (count < NUMBER_OF_QUESTIONS) {
13             // 1. Generate two random single-digit integers
14             int number1 = (int)(Math.random() * 10);
15             int number2 = (int)(Math.random() * 10);
16
17             // 2. If number1 < number2, swap number1 with number2
18             if (number1 < number2) {
19                 int temp = number1;
20                 number1 = number2;
21                 number2 = temp;
22             }
23         }
```



Subtraction Quiz Program

```
24 // 3. Prompt the student to answer "What is number1 - number2?"
25 System.out.print(
26     "What is " + number1 + " - " + number2 + "? ");
27 int answer = input.nextInt();
28
29 // 4. Grade the answer and display the result
30 if (number1 - number2 == answer) {
31     System.out.println("You are correct!");
32     correctCount++; // Increase the correct answer count
33 }
34 else
35     System.out.println("Your answer is wrong.\n" + number1
36         + " - " + number2 + " should be " + (number1 - number2));
37
38 // Increase the question count
39 count++;
40
41 output += "\n" + number1 + "-" + number2 + "=" + answer +
42     ((number1 - number2 == answer) ? " correct" : " wrong");
43 }
44
45 long endTime = System.currentTimeMillis();
46 long testTime = endTime - startTime;
47
48 System.out.println("Correct count is " + correctCount +
49     "\nTest time is " + testTime / 1000 + " seconds\n" + output);
50 }
51 }
```



Subtraction Quiz Program

♦ Run the Program:

```
What is 9 - 2? 7 Enter
You are correct!

What is 3 - 0? 3 Enter
You are correct!

What is 3 - 2? 1 Enter
You are correct!

What is 7 - 4? 4 Enter
Your answer is wrong.
7 - 4 should be 3

What is 7 - 5? 4 Enter
Your answer is wrong.
7 - 5 should be 2

Correct count is 3
Test time is 1021 seconds

9-2=7 correct
3-0=3 correct
3-2=1 correct
7-4=4 wrong
7-5=4 wrong
```

[Click here](#) to view and trace code



GCD Program

- ♦ Write a program to ask the user to enter two positive integers. You should then find the greatest common divisor (GCD) and print the result to the user.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



GCD Program

♦ Step 1: Problem-solving Phase

- GCD

- ♦ $\text{GCD}(4,2) = 2$

- ♦ $\text{GCD}(16,24) = 8$

- ♦ $\text{GCD}(25, 60) = 5$

- So how do you calculate the GCD?

- Are you ready to code?

- ♦ NO!

- Always, first think about the problem

- And understand the solution 200% before coding!



GCD Program

♦ Step 1: Problem-solving Phase

– $\text{GCD}(n1, n2)$

- ♦ You know that the number 1 is a common divisor
 - because 1 divides into everything
- ♦ But is 1 the greatest common divisor?
- ♦ So you can check the next values, one by one
 - Check 2, 3, 4, 5, 6, ...
 - Keep checking all the way up to the smaller of $n1$ or $n2$
- ♦ Whenever you find a new common divisor, this becomes the new **gcd**
- ♦ After you check all the possibilities, the value in the variable **gcd** is the GCD of $n1$ and $n2$



GCD Program

♦ Step 2: Implementation

LISTING 5.9 GreatestCommonDivisor.java

```
1  import java.util.Scanner;
2
3  public class GreatestCommonDivisor {
4      /** Main method */
5      public static void main(String[] args) {
6          // Create a Scanner
7          Scanner input = new Scanner(System.in);
8
9          // Prompt the user to enter two integers
10         System.out.print("Enter first integer: ");
11         int n1 = input.nextInt();
12         System.out.print("Enter second integer: ");
13         int n2 = input.nextInt();
14
15         int gcd = 1; // Initial gcd is 1
16         int k = 2; // Possible gcd
17         while (k <= n1 && k <= n2) {
18             if (n1 % k == 0 && n2 % k == 0)
19                 gcd = k; // Update gcd
20             k++;
21         }
22
23         System.out.println("The greatest common divisor for " + n1 +
24             " and " + n2 + " is " + gcd);
25     }
26 }
```



GCD Program

♦ Run the Program:



Enter first integer: 125

Enter second integer: 2525

The greatest common divisor for 125 and 2525 is 25

[Click here](#) to view and trace code



Future Tuition Program

- ◆ A university charges \$10,000 per year for study (tuition). The cost of tuition increases 7% every year. Write a program to determine how many years until the tuition will increase to \$20,000.
- ◆ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Future Tuition Program

♦ Step 1: Problem-solving Phase

– THINK:

- ♦ How do we solve this on paper?
 - Cost of Year0: \$10,000
 - Cost of Year1: $\text{Year0} * 1.07$
 - Cost of Year2: $\text{Year1} * 1.07$
 - Cost of Year3: $\text{Year2} * 1.07$
 - ...
- ♦ So keep computing the tuition until it is at least \$20,000
- ♦ Once you get to \$20,000, print the number of years taken



Future Tuition Program

♦ Step 1: Problem-solving Phase

– THINK:

- ◆ Now a closer look at some of the code:

```
double tuition = 10000; int year = 0;  
tuition = tuition*1.07; year++;  
tuition = tuition*1.07; year++;  
tuition = tuition*1.07; year++;  
...
```

- ◆ So we would keep doing this until **tuition** is greater than or equal to \$20,000
- ◆ Then, at that point, we print the value in variable **year**
- ◆ How to do this? Use a **while** loop!



Future Tuition Program

♦ Step 2: Implementation

LISTING 5.10 FutureTuition.java

```
1  public class FutureTuition {
2      public static void main(String[] args) {
3          double tuition = 10000;    // Year 0
4          int year = 0;
5          while (tuition < 20000) {
6              tuition = tuition * 1.07;
7              year++;
8          }
9
10         System.out.println("Tuition will be doubled in "
11             + year + " years");
12         System.out.printf("Tuition will be $%.2f in %ld years",
13             tuition, year);
14     }
15 }
```



Future Tuition Program

♦ Run the Program:

```
Tuition will be doubled in 11 years  
Tuition will be $21048.52 in 11 years
```



[Click here](#) to view and trace code



Guessing Game Program (v2)

- ♦ Write a program to guess a number (just like Program 2 in these slides). But this time, you should use a **break** statement to break out of the loop, and the loop condition should be **true**.
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Guessing Game Program (v2)

LISTING 5.3 GuessNumber.java

```
1  import java.util.Scanner;
2
3  public class GuessNumber {
4      public static void main(String[] args) {
5          // Generate a random number to be guessed
6          int number = (int)(Math.random() * 101);
7
8          Scanner input = new Scanner(System.in);
9          System.out.println("Guess a magic number between 0 and 100");
10
11         int guess = -1;
12         while (guess != number) {
13             // Prompt the user to guess the number
14             System.out.print("\nEnter your guess: ");
15             guess = input.nextInt();
16
17             if (guess == number)
18                 System.out.println("Yes, the number is " + number);
19             else if (guess > number)
20                 System.out.println("Your guess is too high");
21             else
22                 System.out.println("Your guess is too low");
23         } // End of loop
24     }
25 }
```



```
import java.util.Scanner;

public class GuessNumberUsingBreak {
    public static void main(String[] args) {
        // Generate a random number to be guessed
        int number = (int)(Math.random() * 101);

        Scanner input = new Scanner(System.in);
        System.out.println("Guess a magic number between 0 and 100");

        while (true) {
            // Prompt the user to guess the number
            System.out.print("\nEnter your guess: ");
            int guess = input.nextInt();

            if (guess == number) {
                System.out.println("Yes, the number is " + number);
                break;
            }
            else if (guess > number)
                System.out.println("Your guess is too high");
            else
                System.out.println("Your guess is too low");
        } // End of loop
    }
}
```

[Click here](#) to view and trace code



Palindromes Program

- ♦ Write a program that prompts the user to enter a string and then reports whether the string is a palindrome.
 - Note: a palindrome is a string that reads the same forwards and backwards
 - Examples: "madam", "racecar", "noon"
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Palindromes Program

♦ Step 1: Problem-solving Phase

– THINK:

- ♦ How can we solve this?
- ♦ Consider the word "**racecar**"
- ♦ Solution:
 - Compare the first letter to the last letter
 - compare the "**r**" to the "**r**"
 - Then compare the second letter to the second to last letter
 - compare the two "**a**" letters
 - Keep doing this process until a mismatch is found OR all the characters in the string have been checked
 - One exception is if the string has an odd number of characters, in which case the middle character will be remaining



Palindromes Program

LISTING 5.14 Palindrome.java

```
1  import java.util.Scanner;
2
3  public class Palindrome {
4      /** Main method */
5      public static void main(String[] args) {
6          // Create a Scanner
7          Scanner input = new Scanner(System.in);
8
9          // Prompt the user to enter a string
10         System.out.print("Enter a string: ");
11         String s = input.nextLine();
12
13         // The index of the first character in the string
14         int low = 0;
15
16         // The index of the last character in the string
17         int high = s.length() - 1;
18     }
```



Palindromes Program

```
18
19     boolean isPalindrome = true;
20     while (low < high) {
21         if (s.charAt(low) != s.charAt(high)) {
22             isPalindrome = false;
23             break;
24         }
25
26         low++;
27         high--;
28     }
29
30     if (isPalindrome)
31         System.out.println(s + " is a palindrome");
32     else
33         System.out.println(s + " is not a palindrome");
34 }
35 }
```



Program 10: Palindromes

♦ Run the Program:



Enter a string: noon ↵ Enter
noon is a palindrome



Enter a string: moon ↵ Enter
moon is not a palindrome

[Click here](#) to view and trace code



Prime Numbers Program

- ♦ Write a program to display the first 50 **prime numbers** in five lines (so 10 numbers per line).
 - Note: any integer greater than 1 is **prime** if it can only be divided by **1** or **itself**.
 - Example:
 - ♦ **2**, **3**, **5**, and **7** are prime numbers
 - ♦ **4**, **6**, **8**, and **9** are not prime numbers
- ♦ Remember:
 - Step 1: Problem-solving Phase
 - Step 2: Implementation Phase



Prime Numbers Program

♦ Step 1: Problem-solving Phase

– THINK:

- ♦ How can we solve this?
- ♦ We need to check EACH integer greater than 1
 - so start at 2, then 3, then 4, then 5, ...
- ♦ And for EACH of those integers, we need to check if it is prime
- ♦ IF it is prime, we need to increase our count
 - because we found a new prime number
- ♦ **and** we also need to print it to the screen
 - but we can only print 10 per line
 - so we need to consider how many have been printed already



Prime Numbers Program

♦ Step 1: Problem-solving Phase

– THINK:

- ♦ So we need a loop!
- ♦ How many times will we loop?
 - MANY times
 - Because we are checking EACH integer greater than 1 to determine if it is a prime number
- ♦ So will the loop go on for infinity?
 - No!
- ♦ So for how long will the loop run?
 - Until we find and print **50** prime numbers!
 - Guess what: we now have our *loop-continuation-condition*!



Prime Numbers Program

♦ Step 1: Problem-solving Phase

– Draft of code:

```
final int NUMBER_OF_PRIMES = 50;
int count = 0;
int number = 2; // check all numbers starting at 2

while (count < NUMBER_OF_PRIMES) {
    // Test if the number is prime

    if (number is prime) {
        // Print the number to the screen
        // increase count
    }

    // Check the next number
    Increment number by 1;
}
```



Prime Numbers Program

♦ Step 1: Problem-solving Phase

– THINK:

- ♦ Given a number, how can we determine if it is prime?
- ♦ Check if it is divisible by **2**, **3**, **4**, ..., **number/2**
 - If any of those values evenly divide number, then it is not prime
- ♦ So we use a **for** loop (from **2** until **number/2**)
- ♦ Example: consider the number 11
 - 2 does not divide into 11
 - 3 does not divide into 11
 - 4 does not divide into 11
 - 5 does not divide into 11
 - *Therefore, 11 is prime!*

The **for** loop ends at **number/2**. Why? Because any value larger than $\frac{1}{2}$ the original number certainly cannot divide evenly into the number!



Prime Numbers Program

♦ Step 1: Problem-solving Phase

– Draft of inner **for** loop:

```
// Use a boolean variable isPrime and set initially to true
boolean isPrime = true;

for (int divisor = 2; divisor <= number/2; divisor++) {
    // IF divisor evenly divides into number, number is NOT prime
    if (number % divisor == 0) {
        isPrime = false;
        Exit/break out of loop!
    }
}
```



Prime Numbers Program

♦ Step 2: Implementation

LISTING 5.15 PrimeNumber.java

```
1  public class PrimeNumber {
2      public static void main(String[] args) {
3          final int NUMBER_OF_PRIMES = 50; // Number of primes to display
4          final int NUMBER_OF_PRIMES_PER_LINE = 10; // Display 10 per line
5          int count = 0; // Count the number of prime numbers
6          int number = 2; // A number to be tested for primeness
7
8          System.out.println("The first 50 prime numbers are \n");
9
10         // Repeatedly find prime numbers
11         while (count < NUMBER_OF_PRIMES) {
12             // Assume the number is prime
13             boolean isPrime = true; // Is the current number prime?
14
15             // Test whether number is prime
16             for (int divisor = 2; divisor <= number / 2; divisor++) {
17                 if (number % divisor == 0) { // If true, number is not prime
18                     isPrime = false; // Set isPrime to false
19                     break; // Exit the for loop
20                 }
21             }
22         }
```



Program 11: Prime Number

♦ Step 2: Implementation

```
23      // Display the prime number and increase the count
24      if (isPrime) {
25          count++; // Increase the count
26
27          if (count % NUMBER_OF_PRIMES_PER_LINE == 0) {
28              // Display the number and advance to the new line
29              System.out.println(number);
30          }
31          else
32              System.out.print(number + " ");
33      }
34
35      // Check if the next number is prime
36      number++;
37  }
38  }
39  }
```

Prime Numbers Program

♦ Run the Program:



The first 50 prime numbers are

2 3 5 7 11 13 17 19 23 29

31 37 41 43 47 53 59 61 67 71

73 79 83 89 97 101 103 107 109 113

127 131 137 139 149 151 157 163 167 173

179 181 191 193 197 199 211 223 227 229

[Click here](#) to view and trace code